

```
(self.x, self.y),  
(self.x + self.width, self.y)))
```

Listing 9-4: The `Triangle` class

To understand the polymorphism at work here, let's look at the code of the `clickedInside()` method for each shape. The `clickedInside()` method of the `Square` class was very simple: a check to see if the mouse click occurred inside the rectangle of the `Square`. The details of the calculations for `clickedInside()` in the `Circle` and `Triangle` classes are not particularly important, but they clearly are doing different calculations. The `clickedInside()` method of the `Circle` class ❶ only reports a click if the user clicks on a colored pixel of the shape. That is, it detects a click that is within the bounding rectangle of the circle, but the click must also be within the radius of the circle for it to count as a click. The `clickedInside()` method of the `Triangle` class ❷ must determine if the user has clicked on a pixel inside the colored triangular portion of the rectangle. The methods in all three classes accept a mouse click as a parameter and return either `True` or `False` as a result.

The `getArea()` ❸ ❹ and `draw()` methods ❺ ❻ of these classes have names that are identical to methods of the `Square` class, but they do different work internally. There's a different calculation for area, and they draw different shapes.

The Main Program Creating Shapes

[*Listing 9-5*](#) shows the source of the main program, which creates a list of randomly chosen shape objects.

File: `Shapes/Main_ShapesExample.py`

```
import pygame  
import sys  
from pygame.locals import *  
from Square import *  
from Circle import *  
from Triangle import *  
import pygwidgets
```